LAB 4: LoRa Point-to-Point (P2P) and Arduino MEGA Integration

José Francisco Paca Sotero, Daniel Ignacio Casquino Paz,
Nicolas Stigler Yañez, Sofia Valentina Ku Paredes
Universidad de Ingeniería y Tecnología

I. Introduction

THIS laboratory studies a LoRa (long range) point-to-point (P2P) link, its received signal measurements and its integration with an Arduino MEGA. The objectives are to configure the radios through serial commands, exchange messages and examine the received signal strength indicator (RSSI) and signal-to-noise ratio (SNR) over time.

A. Theoretical Framework

LoRa uses chirp spread spectrum modulation for low-data-rate radio communication. The long range wide area network (LoRaWAN) protocol adds network and medium-access functions to the LoRa physical layer. A direct P2P link between two radios therefore differs from a LoRaWAN deployment with gateways and a network server [1].

The spreading factor (SF) and bandwidth (BW) set the symbol duration,

$$T_{\mathrm{sym}} = \frac{2^{SF}}{BW}, \tag{1}$$

where BW is expressed in hertz. Increasing SF at a fixed bandwidth makes symbols longer. Coding rate (CR) and payload length also affect the time required to transmit a packet [1]. For the laboratory example $SF = 7$ and $BW = 125$ kHz, Equation 1 gives $T_{\mathrm{sym}} = 1.024$ ms. This is a calculated symbol duration, rather than a measured packet duration.

RSSI describes received signal level in dBm, relative to 1 mW. SNR compares signal power with noise power and is expressed in dB. Negative SNR means the estimated signal power is below the estimated noise power. The receiver firmware uses RAKwireless Unified Interface 3 (RUI3), the firmware development platform from radio manufacturer RAKwireless, and reports RSSI, SNR and hexadecimal payload in each P2P receive event [2].

A universal asynchronous receiver/transmitter (UART) carries serial commands between the computer or microcontroller and the radio. The commands use the attention prefix `AT` [2]. With `AT+PSEND`, each payload byte is represented by two hexadecimal digits. A local `OK` acknowledges the command, while `TXP2P DONE` signals completion of transmission. Reception must be checked separately at the other radio [2].

B. State of the Art

Augustin *et al.* examined LoRa's physical and data-link behavior through field tests and simulation. Their study helps relate spreading factor to radio performance and distinguish the modulation from the LoRaWAN network protocol [1].

Bor *et al.* combined experimental models with simulation to examine how shared-medium LoRa networks scale. They identified communication-parameter selection and multiple sinks as ways to improve scalability [3]. Here, the scope is a single P2P receiver record, so its RSSI and SNR describe that link rather than network capacity.

More recently, Lorincz *et al.* examined how transmission settings and physical obstacles affect indoor RSSI and SNR in a LoRaWAN test platform [4]. Their scope is broader than the present P2P trace, but it shows why propagation conditions should be recorded alongside radio measurements.

II. Methodology

A. Components and Functional Arrangement

The laboratory guide specifies RAK3172 radio modules from RAKwireless on RAK3272S breakout boards, with antennas and universal serial bus (USB) to transistor-transistor logic (TTL) adapters for serial control. Figure 1(a) shows these functional roles: commands reach each radio through UART, while messages travel over the LoRa link. Serial terminals display command responses and reception events. The serial transmit (TX) and receive (RX) paths carry commands and responses. This arrangement supports the AT verification and P2P experiments [5].

For the counter transmitter, the Arduino MEGA2560 replaces the manual command source, using `Serial1` (serial interface 1) to communicate with the RAK and USB serial to print diagnostic output. Figure 1(b) separates this transmitter path from the two-radio experiment. The assembly photograph in Figure 9 shows the MEGA, radio, antenna, breadboard, jumper wires and resistors.

The guide requires voltage adaptation at the RAK receive input because the MEGA uses 5 V logic and the RAK uses 3.3 V logic [5]. The three-resistor divider adapts the UART signal level; its nominal output voltage is derived in the physical implementation section.

B. Radio Configuration and Counter Program

The RUI3 P2P configuration includes frequency, spreading factor, bandwidth, coding rate, preamble length and transmit power [2].

Frequency, modulation settings and other packet-format settings must be compatible between transmitter and receiver.

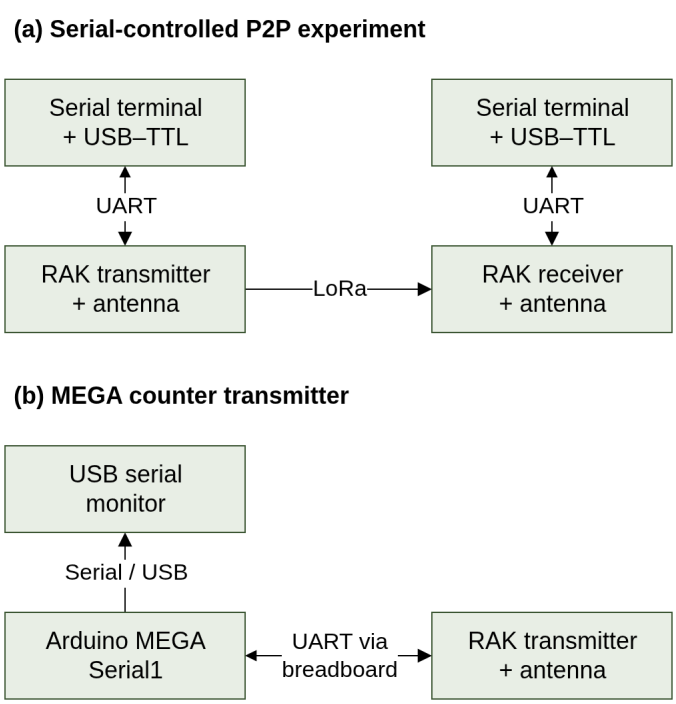


Figure 1. Functional arrangement of (a) the serial-controlled P2P experiment specified by the guide and (b) the MEGA counter transmitter documented by the sketch and assembly photograph. Arrows identify communication paths, not individual wires.

Transmit power describes the transmitting radio and is not a requirement that both radios use identical output power.

Figure 2 traces the shared counter sketch, whose structure follows Appendix H [5]. Initialization opens both serial interfaces at 115200 baud, then sends the version queries, P2P mode and link configuration, and transmitter-mode command. The repeated loop increments the counter, encodes its decimal characters as hexadecimal bytes using the American Standard Code for Information Interchange (ASCII) and calls `sendRAK` to send `AT+PSEND`. The shared sketch and the code visible in the demonstration video use different spreading factors, as detailed in the implementation discussion.

The `sendRAK` function waits until at least one UART byte is available, reads the bytes currently available and prints the collected text. It does not parse a success response or wait specifically for `TXP2P DONE`; it also has no timeout on the initial wait. The 5000 ms delay occurs after this function returns, so it specifies a programmed pause rather than a measured transmission period or a reception guarantee.

C. Receiver Record Processing

The analysis uses 108 previously recorded receiver entries. The input contains reception timestamps, RSSI and SNR. The elapsed time for sample i is

$$\tau_i = t_i - t_1. \tag{2}$$

The original order and every numeric observation are retained. Figure 3 summarizes this processing sequence. The Python script produces a numeric comma-separated values (CSV) file, summary statistics and two plots with a shared time axis. Each

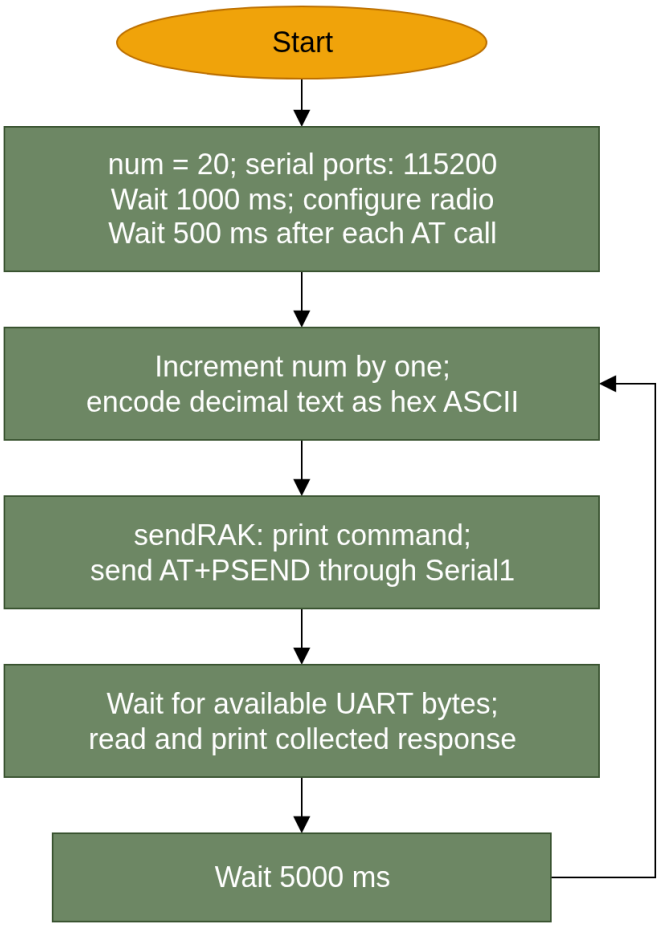


Figure 2. Counter-program flow. The `sendRAK` call sends the command and collects available response bytes; it does not verify their contents or remote reception.

point represents a received observation, without smoothing or interpolation.

D. Interpretation of the Measurements

The timestamps describe when packets were recorded at the receiver. Intervals between received entries are not necessarily the transmitter's configured period.

III. Development and Discussion

The RAK modules responded to serial AT commands and exchanged hexadecimal payloads over the LoRa P2P link. The Arduino MEGA then automated radio configuration and counter transmission, while the receiver output showed decoded messages, signal measurements and reception errors during the range test.

A. AT Command Verification

We tested the serial communication between the computer and the RAK module using AT commands. The module was connected through a USB-to-TTL converter, and we opened the Arduino integrated development environment (IDE) Serial Monitor at 115200 baud to send the commands and observe the responses.

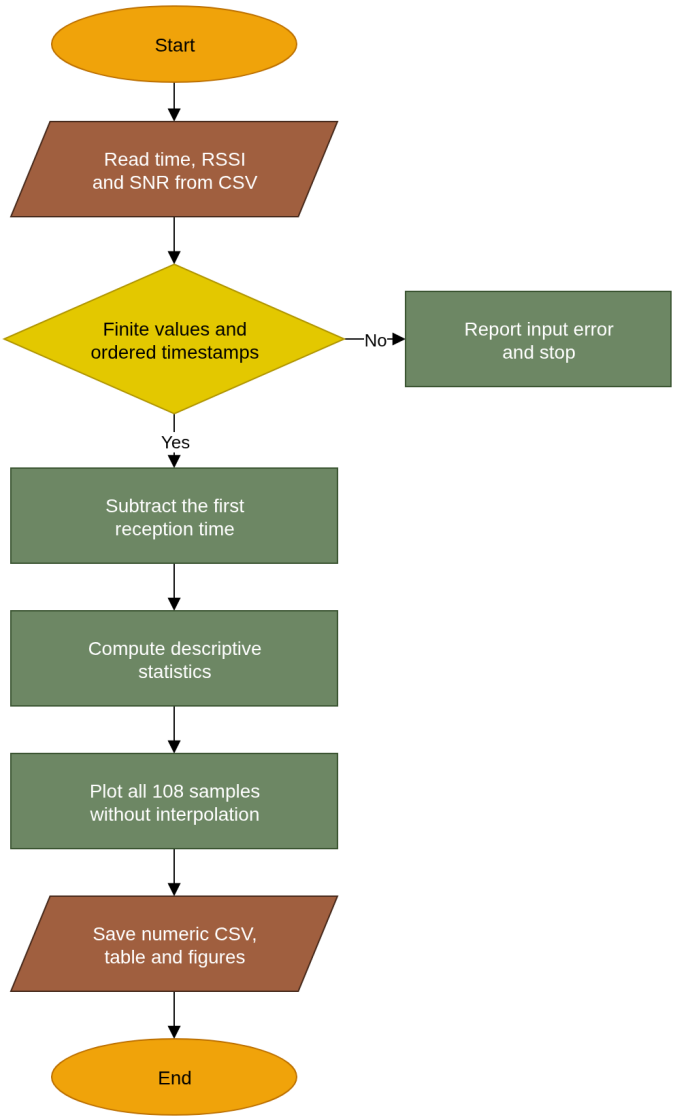


Figure 3. Offline processing sequence for the receiver record.

The commands used included `AT` to check serial responsiveness, `ATE` to enable command echo, and `ATZ` to reset the module. We used `AT+VER=?` to identify the firmware version and `AT+SN=?` to query the device serial number.

Figure 4 captures part of the command-and-response session. The module reported firmware version `RUI_4.2.1_RAK3172-E`, displayed its configured LoRa P2P mode, and returned `OK` responses to serial commands.

B. LoRa P2P Communication

We established a point-to-point communication between two RAK modules, configuring one as the sender and the other as the receiver. Both modules were connected to their respective computers through USB-to-TTL converters, allowing us to configure and monitor them using AT commands.

First, we used `AT+NWM=0` to enable LoRa P2P mode on both devices. We then configured the communication parameters using `AT+P2P`, making sure that both modules shared the same frequency, spreading factor, bandwidth, coding

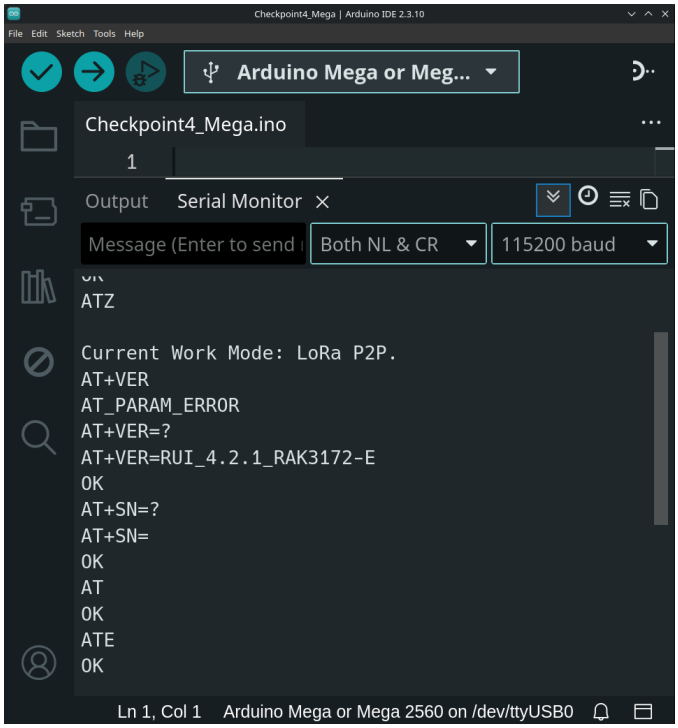


Figure 4. Arduino IDE Serial Monitor showing AT command responses, firmware identification, and the configured LoRa P2P mode.

rate, and preamble length. The receiver was configured with `AT+PRECV=65534` to listen continuously, while the sender used `AT+PRECV=0` before transmitting. To send a message, we used the command `AT+PSEND`, followed by the payload encoded in hexadecimal. The hexadecimal representation is required by the module's AT interface, so the original text must be converted before transmission.

After configuring both RAK modules, we verified their responses through the Arduino IDE Serial Monitor. Both terminals responded to the AT commands and confirmed that the devices were operating in LoRa P2P mode. Figure 5 shows the recorded output from the sender terminal, including several `OK` responses and `+EVT:TXP2P DONE` events, which indicate completed transmission operations. Although the same configuration procedure was performed on both devices, only the sender terminal was photographed. The receiver's output was preserved in a text file and is presented later in this section.

Figure 5 shows the sender's serial monitor displaying several `+EVT:TXP2P DONE` events. These messages indicate that the module completed its transmission operations. However, they do not independently confirm reception, which must be verified through the receiver's output.

The physical connection used for the communication test is shown in Figure 6. The USB-to-TTL converter allowed us to exchange commands and responses with the RAK module through UART, while the antenna was used for wireless transmission.

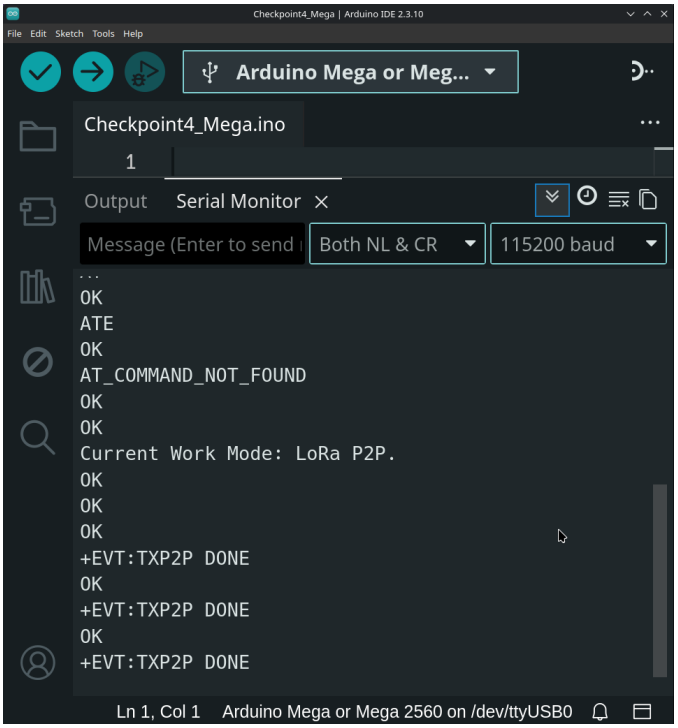


Figure 5. RAK module Serial Monitor showing LoRa P2P mode, command responses, and completed transmission events.

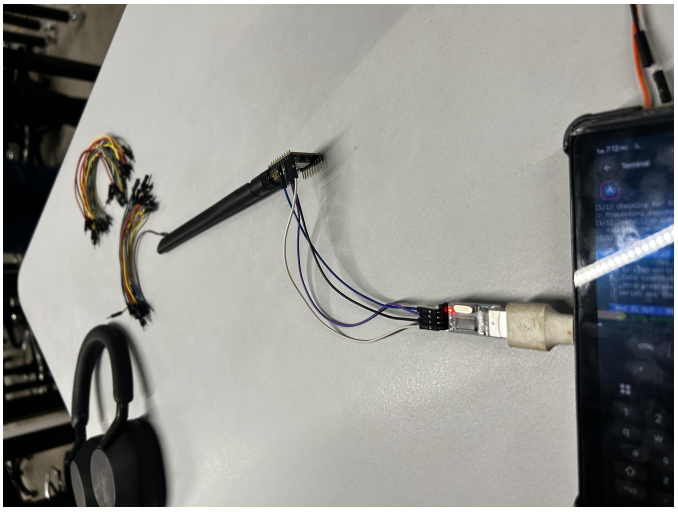


Figure 6. RAK module connected through a USB-to-TTL converter for LoRa P2P communication.

Although we did not capture a photograph of the receiver's serial monitor, we saved its output in a text file. The recorded results included the following messages:

```
OK
OK
RAKwireless RAK3172-E
\------------------------------------------------------
Version: RUI_4.2.1_RAK3172-E
Current Work Mode: LoRa P2P.
OK
OK
+EVT:RXP2P:-42:8:43
+EVT:RXP2P:-40:8:67
+EVT:RXP2P:-37:8:3637207061706F69
```

These messages follow the format `+EVT:RXP2P:RSSI:SNR:PAYLOAD`, where RSSI represents the received signal strength, SNR indicates the signal-to-noise ratio, and the final field contains the hexadecimal payload. The recorded RSSI values were -42, -40, and -37 dBm, while the SNR remained at 8 dB for all three messages. These values confirm that the receiver was detecting the transmissions and reporting their signal characteristics.

The first two received messages were test transmissions, while the final message contained the intended text encoded in hexadecimal. The received payload for the last message was `3637207061706F69`. To verify its contents, we converted it from hexadecimal to text, obtaining the message `67 papoi`, as shown in Figure 7. This confirms that the received bytes could be decoded into the intended message.

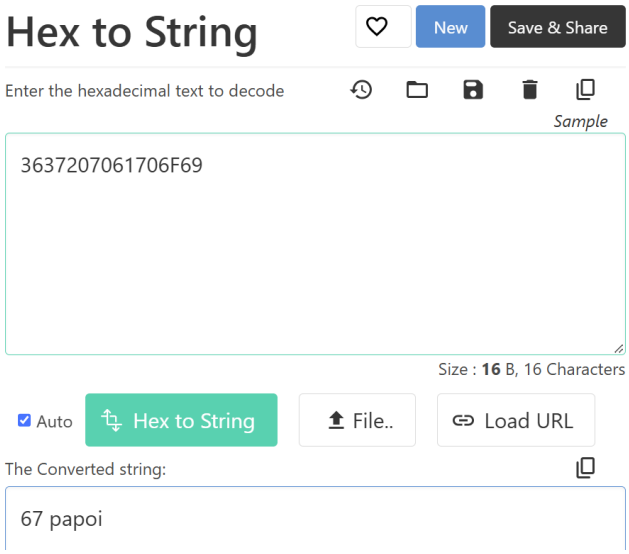


Figure 7. Conversion of the received hexadecimal payload into the original text message "67 papoi".

Overall, the results demonstrate successful communication between the two RAK modules. The sender reported completed transmissions, while the receiver registered incoming packets containing signal information and hexadecimal payloads. This allowed us to verify both the wireless communication and the representation of the transmitted data.

The range test was later continued during the automated transmissions with the Arduino MEGA. The receiver was moved progressively farther from the sender while its received messages and signal measurements were monitored. Figures 13 and 14 document the successful receptions and the reception error observed during that demonstration.

1) Receiver Measurements: The record spans 875.078 s, approximately 14.6 minutes. Table I summarizes the received samples. The RSSI arithmetic mean is calculated over the reported dBm values and is not an average received power in milliwatts.

Figure 8 shows the variation of RSSI and SNR over elapsed reception time. Separate panels keep their units distinct. The

Table I
DESCRIPTIVE STATISTICS OF THE RECEIVED-SAMPLE RECORD.

Received samples	108
Duration (s)	875.078
RSSI min/max (dBm)	-116 / -22
Mean RSSI reading (dBm)	-86.06
SNR min/max (dB)	-19 / 9
Mean SNR reading (dB)	-2.89

RSSI ranges from -116 to -22 dBm, while SNR ranges from -19 to 9 dB. Both values change throughout the record.

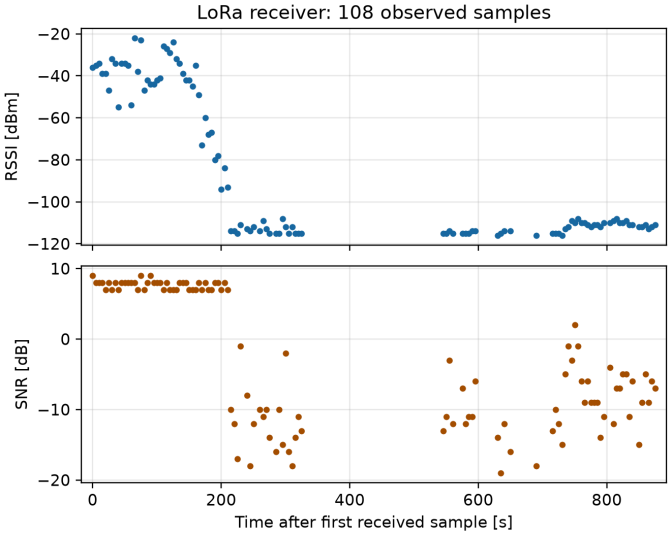


Figure 8. RSSI and SNR for 108 received samples, with time relative to the first reception. Markers show individual observations.

The low-SNR entries are packets that reached the receiver despite their reported signal-to-noise conditions. They do not show how many other transmissions were missed. Empty intervals in the graph contain no recorded observations and cannot, by themselves, establish packet loss.

During the range test, the receiver was progressively moved away from the transmitter while incoming messages were monitored. Figure 8 shows considerable variations in RSSI and SNR throughout the recorded interval, which are consistent with changes in the radio propagation conditions during the experiment.

However, these variations cannot be attributed exclusively to distance, since obstacles, antenna orientation, and interference may also affect the received signal. Furthermore, the receiver's position was not recorded for each sample, so the graph cannot establish an exact relationship between distance and signal strength or determine the maximum communication range.

C. Arduino MEGA Integration and Data Transmission

We integrated a RAK LoRa module with an Arduino MEGA to automate the transmission of messages through LoRa P2P communication. The Arduino was programmed to configure the RAK module and periodically send messages without requiring manual intervention.

1) Physical Implementation: The circuit was assembled using an Arduino MEGA, a RAK module, a breadboard, jumper wires, and three 10 kΩ resistors, as shown in Figure 9 and Figure 10. The Arduino communicates with the RAK module through UART, using its `Serial1` interface connected to the module's UART2 pins. Since the Arduino MEGA operates with 5 V logic and the RAK module uses 3.3 V logic, we implemented a voltage divider using three 10 kΩ resistors connected in series. One resistor was placed between the Arduino TX pin and the RAK RX connection, while the other two resistors were connected between this point and ground (GND). For a nominal 5 V input, the voltage-divider relation in Equation 3 gives an output of approximately 3.33 V.

$$V_{\text{out}} = 5\left(\frac{20\,k\Omega}{10\,k\Omega + 20\,k\Omega}\right) \approx 3.33\,V \tag{3}$$

The voltage divider was necessary to adapt the Arduino's output to the RAK module's logic level and avoid exposing its RX pin directly to 5 V. Both devices also shared a common ground reference to allow UART communication.

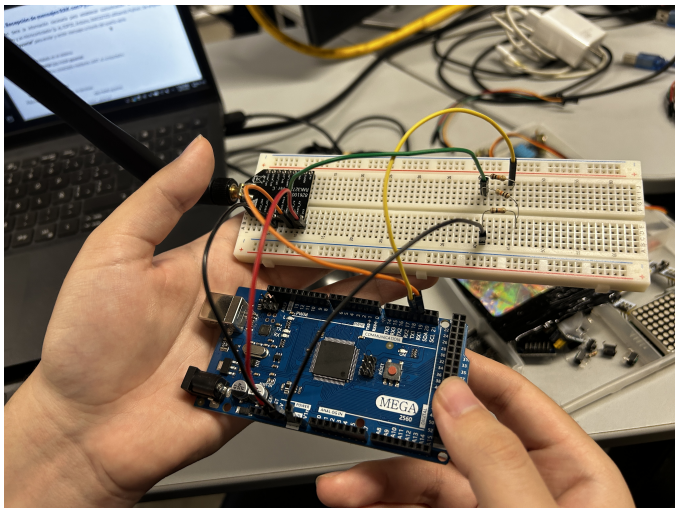


Figure 9. Physical implementation of the Arduino MEGA and RAK LoRa module connected through a breadboard.

2) Arduino Code and Message Transmission: After assembling the circuit, we developed an Arduino program to configure the RAK module and send messages automatically. The program initializes both `Serial` and `Serial1` at 115200 baud.

The first interface is used to display information on the computer, while the second sends AT commands directly to the RAK module. During initialization, the Arduino sends `AT+VER=?` to request the firmware version and `AT+NWM=0` to activate LoRa P2P mode. It then configures the communication parameters using the following command:

```
AT+P2P=923700000:12:125:0:10:14
```

This configuration establishes a frequency of 923.7 MHz, a spreading factor of 12, a bandwidth of 125 kHz, a coding rate of 4/5, a preamble length of 10, and a transmission power of 14 dBm. Finally, `AT+PRECV=0` prepares the module for transmission.

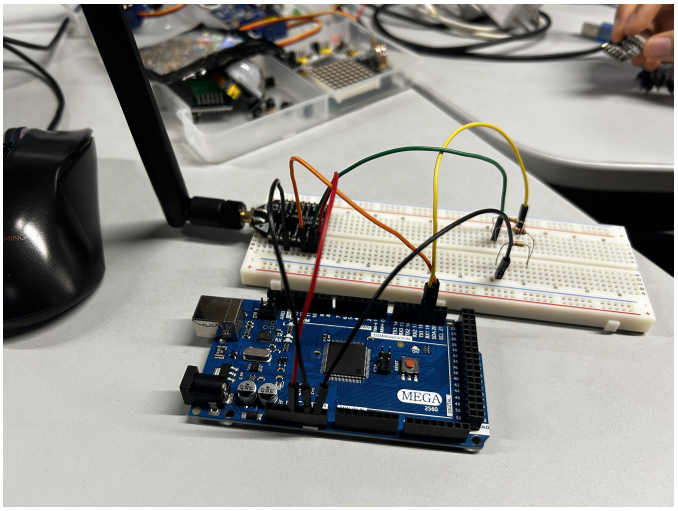


Figure 10. Additional view of the Arduino MEGA and RAK module wiring, including the voltage divider used for UART communication.

In the main loop, the Arduino increments a counter starting from 20, converts the resulting number into its hexadecimal representation, and sends it using `AT+PSEND`. The first transmitted counter value is 21. A delay of 5000 ms follows the response-reading function; it is a programmed pause, rather than a measured transmission period. The program also displays the commands sent and the responses received from the RAK module, allowing us to monitor its execution through the Arduino Serial Monitor.

The shared Arduino sketch is available as `lora.txt` in Drive. It generates and transmits an incrementing counter. The payload represents generated counter data; the sensor-reading stage requested in Section 3.2 of the guide is not demonstrated by this test.

The shared sketch configures spreading factor 12. The sender demonstration shows `AT+P2P=923700000:10:125:0:10:14`, using spreading factor 10. These are two source-code settings, rather than parameter readbacks from both radios; neither can be assigned to the entire 108-sample receiver record.

The `sendRAK` function waits for at least one UART byte and prints the bytes available at that moment. It has no timeout and does not parse `OK` or wait specifically for `TXP2P DONE`. Completion and reception therefore need to be evaluated from the recorded radio output.

Figure 11 shows the Arduino Serial Monitor sending consecutive hexadecimal payloads, including `3332`, `3333`, and `3334`, corresponding to the text values 32, 33, and 34. The `OK` responses and `+EVT:TXP2P DONE` events show that these transmission operations completed on the sender side.

The sender demonstration video shows the Arduino MEGA periodically issuing LoRa transmission commands while the receiver begins moving away to test the communication range. The Serial Monitor displays the hexadecimal payloads, the corresponding `OK` responses, and the `+EVT:TXP2P DONE` events, confirming that the sender continued completing transmission operations during the experiment.

Sender demonstration: Video sender demonstration

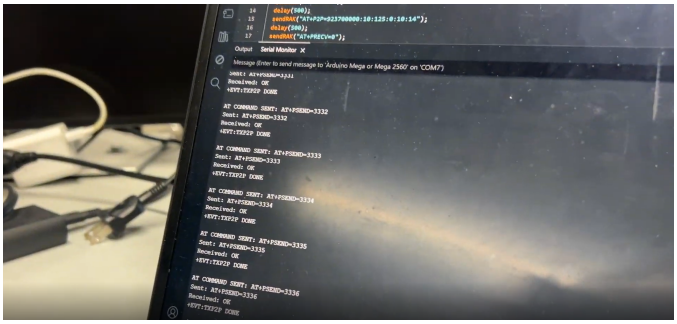


Figure 11. Arduino MEGA Serial Monitor showing consecutive LoRa transmission commands and transmission completion events.

Figure 12 documents another portion of the counter test, showing values 144 through 148. For example, `313434` is the hexadecimal encoding of the three ASCII characters in `144`.

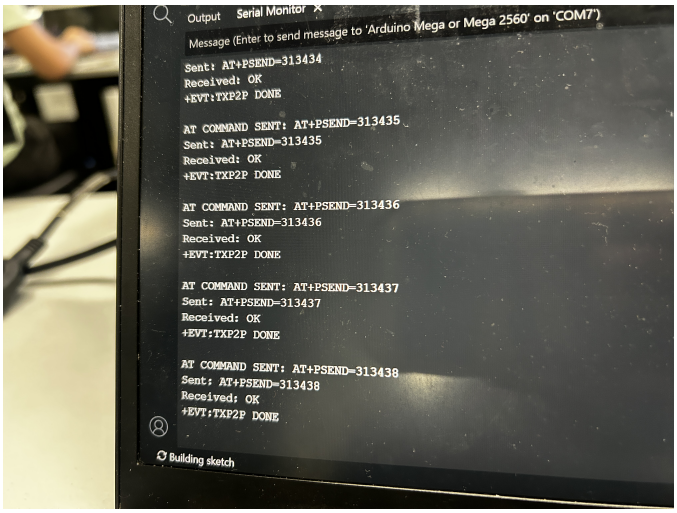


Figure 12. Serial output showing generated counter values 144–148 and local transmission-completion events.

3) Reception and Discussion: To test the automated communication, a second RAK module was connected to another computer and configured as the receiver. Successful receptions demonstrate compatible settings during those exchanges. The receiver was configured for continuous reception using `AT+PRECV=65534`. The receiver was used to monitor the incoming messages and verify the communication between both devices. As shown in Figure 13, it displayed successive received packets and their decoded counter values, together with the corresponding RSSI and SNR measurements. For example, the visible messages include consecutive values from 21 onward, showing that the counter transmitted by the Arduino could be recovered at the receiving end. The recorded RSSI and SNR values varied between messages, illustrating changes in the received signal conditions during the test.

As the receiver was moved farther away, the demonstration eventually showed reception errors, as illustrated in Figure 14. The receiver was then checked using an AT command to confirm that its serial interface was still responding. Recep-

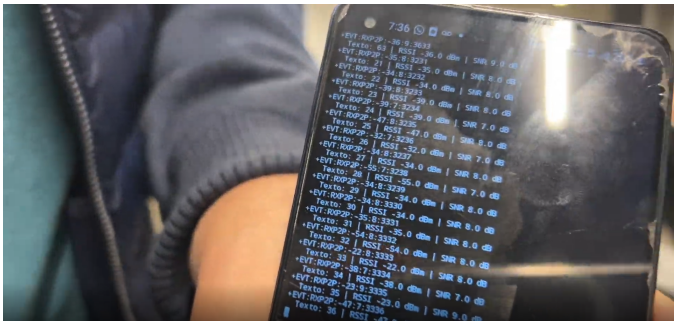


Figure 13. Receiver terminal displaying successive decoded counter values with RSSI and SNR measurements during the LoRa range test.

tion errors occurred during the range test, interspersed with successful receptions; their cause was not isolated.



Figure 14. Receiver terminal showing reception errors after the devices were separated during the range test, followed by a check of the module's operating mode.

The receiving process is documented in two demonstration videos. In the first video, the receiver displays consecutive messages containing the transmitted counter values, together with their corresponding RSSI and SNR measurements. As the receiver moves away from the sender, the incoming messages continue to be monitored, allowing us to observe how the signal conditions change during the experiment.

Receiver demonstration 1: Video receiver demonstration 1

The second video shows a later stage of the range test, when the receiver begins displaying reception errors. An AT command is then executed to verify that the module remains responsive. This check is relevant because it helps distinguish a loss of wireless reception from a complete failure of the module or its serial communication.

Receiver demonstration 2: Video receiver demonstration 2

Together, the videos and Figures 13 and 14 provide evidence of successful periodic reception and the communication difficulties observed as the separation between the devices increased. Although the experiment did not establish an exact maximum distance, it demonstrated that completing a transmission on the sender does not necessarily guarantee reception under all radio conditions.

The Arduino MEGA automated the configuration, hexadecimal conversion, and repeated transmission of messages that had previously been sent manually.

An important aspect of this implementation is that the Arduino does not transmit the numerical value directly as a binary integer. Instead, it converts the number into a text representation and then encodes each character as a hexadecimal byte. This allows the receiving module to obtain the transmitted data in the format expected by the LoRa AT interface.

Additionally, the voltage divider was an essential part of the integration, since successful communication also depends on the electrical compatibility between the devices, not only on using the same serial protocol.

The automated LoRa P2P test demonstrated a practical limitation of the wireless link. The Arduino continued issuing transmission commands, but reliable reception depended on the radio conditions and separation between the modules. This distinction is important for Internet of Things (IoT) applications, where a functioning transmitter alone does not guarantee that data reaches its destination.

Supporting material is available in the LAB4 shared folder.

IV. Conclusions and Recommendations

The serial-command test showed us that successfully communicating with a LoRa module involves more than simply establishing a serial connection. Although the module responded correctly and reported its firmware version and operating mode, the incorrect version command demonstrated how a small syntax error can prevent an instruction from being executed as expected. This helped us understand the importance of interpreting the device's responses rather than assuming that every command was accepted. We recommend verifying the AT command syntax for the installed firmware and checking the responses after each configuration step, especially before proceeding to more complex communication tests. The P2P experiment demonstrated that establishing a LoRa P2P link requires verifying communication from both the transmitter and receiver, since a completed transmission does not necessarily guarantee successful reception. The received messages confirmed that the original information could be recovered from its hexadecimal representation, while the RSSI and SNR measurements showed how signal conditions can vary during communication. The range test also highlighted that a wireless link may become unreliable as the devices move farther apart, although distance alone cannot explain every variation observed. For future tests, we recommend recording the distance and environmental conditions alongside RSSI and SNR, as well as comparing transmitted and received message counts

to better evaluate the reliability and practical range of the communication. The Arduino integration helped us understand how LoRa communication can be automated by integrating a microcontroller with a radio module through UART. Instead of manually entering AT commands, the Arduino MEGA configured the module and periodically transmitted an incrementing counter, whose values could be recovered at the receiver. An important aspect of this implementation was the voltage divider, which allowed us to connect devices operating at different logic levels without directly exposing the RAK input to the Arduino's 5 V output. Additionally, the reception errors observed during the distance test showed that automation alone does not guarantee reliable wireless delivery. We recommend measuring the voltage divider output before connecting the modules and implementing response timeouts and transmission verification in future versions of the code. During our range test, restarting the receiver became difficult once the devices were separated. We therefore also recommend validating the radio configuration at both ends before moving away and preparing a simple way to monitor and restart the receiver remotely.

REFERENCES

[1] A. Augustin, J. Yi, T. Clausen, and W. M. Townsley, "A Study of LoRa: Long Range & Low Power Networks for the Internet of Things," *Sensors*, vol. 16, no. 9, p. 1466, 2016. [Online]. Available: https://doi.org/10.3390/s16091466

[2] *RUI3 AT Command Manual*, RAKwireless, accessed October 23, 2026. [Online]. Available: https://docs.rakwireless.com/product-categories/software-apis-and-libraries/rui3/at-command-manual/

[3] M. Bor, U. Roedig, T. Voigt, and J. Alonso, "Do LoRa Low-Power Wide-Area Networks Scale?" in *Proceedings of the 19th ACM International Conference on Modeling, Analysis and Simulation of Wireless and Mobile Systems (MSWiM)*, 2016, pp. 59–67. [Online]. Available: https://doi.org/10.1145/2988287.2989163

[4] J. Lorincz, K. Levarda, M. Čagalj, and A. Kukuruzović, "A Comprehensive Analysis of LoRa Network Wireless Signal Quality in Indoor Propagation Environments," *Journal of Sensor and Actuator Networks*, vol. 14, no. 6, p. 111, 2025. [Online]. Available: https://doi.org/10.3390/jsan14060111

[5] *LAB 4: LoRa, CS5055, W11–W12*, Universidad de Ingeniería y Tecnología, 2026, laboratory guide.